SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science & Engineering
CO2 - ASSESSMENT TOOL - 2
Reserve Engineering Task

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 2: Reserve Engineering Task**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO2 Statement	*Design effective test cases using black-box, white-box testing techniques and to control methods flow.*		
Student Name	**M Sandhya**	**Reg. No.**	**192312421**
Section/Batch	**Slot A / 2023**	**Date**	**02-11-2026**

BVA-DRIVEN STRESS TESTING STRATEGY FOR A CLOUD FILE SYNCHRONIZATION TOOL

Reverse Engineering of Sync Logic, Equivalence Class Partitioning and Boundary Value Analysis for Lost-File Prevention

1. Introduction and Problem Context

A cloud-based file synchronization tool keeps a local folder and a cloud store identical. It watches the local folder, uploads new or modified files, verifies them, and marks them as synchronized. Users have now reported a recurring problem: some files are “lost”, meaning they never appear in the cloud, appear only partly, or disappear from the sync client without any message. The tool produces user activity logs and a list of “lost file” defect reports, which are the only evidence available about its internal behaviour.

Because the source code is not available, this report treats the tool as a black box. It analyses the defect patterns against three input factors, namely file size, file type and network speed, and works backwards to infer the synchronization logic and the Equivalence Class Partitioning (ECP) that the original developers most probably used. It then reconstructs a revised strategy that combines Boundary Value Analysis (BVA) with stress testing, and justifies how that strategy prevents future data loss.

The central observation is that every lost file sits on or beside a boundary: an empty file, a file exactly at the chunk-size multiple, a file just over the single-upload limit, a file just over the maximum size, or a transfer that runs just beyond the time-out. Such failures are typical of a test design that used one “typical” value per partition and never probed the edges or the combined effect of size and slow networks.

1.1 Objectives

- Identify functional and non-functional requirements of the synchronization tool from the scenario.
- Reverse engineer the probable synchronization algorithm from lost-file patterns related to size, type and network speed.
- Infer the ECP used by the original developers and show which classes were missing.

- Derive positive and negative test cases, boundary conditions and valid and invalid equivalence classes.
- Identify potential defects and explain their impact on users and the business.
- Reconstruct the business rules as a decision table and model the system states, events and valid or invalid transitions.
- Reconstruct a revised testing strategy that integrates BVA with stress testing and justify how it prevents data loss.

1.2 Assumed Defect Evidence

The scenario does not include the actual logs, so the following defect reports are assumed. They are a realistic, simplified sample used to demonstrate the analysis. The thresholds inferred from them (100 MB, 10 MB chunks, 2 GB, 300 s) are hypotheses and must be confirmed against the real logs and the product specification before execution.

Table 1: Assumed lost-file defect reports and activity-log observations

Report ID	File size	File type	Network speed	Observed outcome
DR-01	0 B	.txt	50 Mbps	Lost – never uploaded, no log entry
DR-02	4 KB	.docx	50 Mbps	Synced correctly
DR-03	25 MB	.jpg	20 Mbps	Synced correctly
DR-04	100 MB (exact)	.pdf	25 Mbps	Lost – cloud copy is 90 MB, last chunk missing
DR-05	150 MB	.mp4	2 Mbps	Lost – time-out, no retry, marked complete
DR-06	2.5 GB	.zip	100 Mbps	Lost – silently skipped
DR-07	200 MB (20 × 10 MB)	.iso	50 Mbps	Truncated copy, checksum mismatch
DR-08	12 KB	~budget.xlsx	30 Mbps	Lost – treated as a temporary file
DR-09	3 MB	.png	0.5 Mbps, unstable	Lost – retries exhausted, status not shown
DR-10	80 MB	.pptx	1 Mbps	Lost – time-out after 300 s
DR-11	100 MB − 1 B	.pdf	25 Mbps	Synced correctly
DR-12	105 MB	.zip	25 Mbps	Synced correctly

2. Requirements Analysis

The requirements below are inferred from the scenario and from what a user reasonably expects of a synchronization service. They are the source of the positive and negative test cases in Section 6.

2.1 Functional Requirements

Table 2: Functional requirements

ID	Requirement	Linked defects
FR-01	Detect create, modify, rename and delete events in the watched folder.	DR-08
FR-02	Upload every new or modified file and confirm that the cloud store has received it.	DR-01, DR-05
FR-03	Support every size from 0 B up to the stated maximum (assumed 2 GB), using chunked transfer for large files.	DR-01, DR-04, DR-07
FR-04	Support all permitted file types; exclude only genuine temporary or system files.	DR-08
FR-05	Resume an interrupted upload from the last confirmed chunk instead of restarting or abandoning it.	DR-05, DR-10
FR-06	Verify size and checksum in the cloud before marking a file as synchronized.	DR-04, DR-07
FR-07	Retry failed transfers with back-off up to a configured limit, then report a visible failure.	DR-09
FR-08	Notify the user of every file that is skipped, rejected or failed; never skip silently.	DR-06, DR-09
FR-09	Resolve edit conflicts without overwriting either version.	–
FR-10	Record every state change of every file in the activity log.	All

2.2 Non-Functional Requirements

Table 3: Non-functional requirements

ID	Category	Requirement (assumed targets)
NFR-01	Reliability	Zero silent file loss. Every file is either synchronized and verified, or the user is clearly told why not.
NFR-02	Performance	A file under 1 MB is synchronized within 30 s on a network of 10 Mbps or faster.
NFR-03	Robustness	Operates correctly from 0.25 Mbps to 1 Gbps and survives disconnection and reconnection.
NFR-04	Scalability	Handles at least 100,000 files and 500 concurrent change events without losing any.
NFR-05	Data integrity	100% of synchronized files match the source checksum.
NFR-06	Recoverability	After a crash or restart, the client resumes without duplication or loss.
NFR-07	Security and privacy	Transfers use TLS; logs record identifiers or masked names, not file content.
NFR-08	Usability	The client displays an accurate status for each file (Syncing, Waiting, Synced, Failed, Skipped).

3. Reverse Engineering the Synchronization Algorithm

3.1 Defect-Pattern Analysis

Each pattern in Table 1 points to a specific decision in the original algorithm. Successful reports (DR-02, DR-03, DR-11, DR-12) are as informative as the failures, because they show where the logic behaves correctly.

Table 4: Defect patterns and inferred logic

Observed pattern	Evidence	Inferred logic in the original algorithm
Zero-byte files are never synchronized, at any speed.	DR-01	A guard such as “if size == 0 then return” (or a chunk count of zero so the upload loop never runs) treats empty files as nothing to send.
Files whose size is an exact multiple of the chunk size lose the final chunk; 99.99 MB and 105 MB are fine.	DR-04, DR-07, DR-11, DR-12	Chunk loop stops one iteration early and the “tail” upload runs only when size mod chunk ≠ 0. Exact multiples therefore skip the last full chunk. Chunk size appears to be 10 MB and the chunked path starts at 100 MB.
Large files on slow links disappear; the same files on fast links work.	DR-05, DR-10	A fixed whole-transfer time-out of about 300 s with no resume. Transfer time = size × 8 ÷ speed: 150 MB at 2 Mbps takes 600 s and 80 MB at 1 Mbps takes 640 s, both over the limit.
Files above 2 GB vanish without a message.	DR-06	A maximum size check that returns silently instead of notifying the user.
User files whose names start with “~” are lost.	DR-08	The temporary-file filter matches on a name prefix such as “~” or “.” and is wider than the real lock or temp file patterns.
Failures on a flapping network leave no visible error.	DR-09	After the retry limit, the file is marked “done” and removed from the queue, so the failure is never displayed.
File type alone does not decide success.	DR-02 to DR-12	Type matters only through the name filter (DR-08) and through typical size (video, ISO and archives are large). The real drivers are size and speed.

3.2 Inferred Pseudocode of the Original Logic

The pseudocode below is the simplest algorithm that is consistent with every report in Table 1. Lines marked with comments contain the suspected faults.

```
on file_event(f):
if f.name starts with '~' or '.' or ends with '.tmp': return   # F1: filter too wide
if f.size == 0: return                                  # F2: empty file skipped
if f.size > 2 GB: return                                  # F3: silent skip
if f.size < 100 MB:
ok = upload_single(f, timeout = 300 s)
else:
chunks = ceil(f.size / 10 MB)
for i in 0 .. chunks - 2: upload_chunk(f, i)              # F4: off-by-one
if f.size mod 10 MB != 0: upload_tail(f)                   # F4: exact multiple loses last chunk
ok = (all calls finished within 300 s in total)           # F5: no resume
if ok or retries >= 3:                                  # F6: failure treated as done
mark_synced(f); remove_from_queue(f)                        # F7: no checksum verification
```

3.3 Potential Defects and Their Impact

Table 5: Defects, impact and severity

Fault	Defect	Impact	Severity
F1	Over-broad temporary-file filter	Valid user documents are never backed up; user believes they are protected.	High
F2	Empty files skipped	Placeholder, lock and marker files that applications rely on are missing after restore.	Medium
F3	Silent skip above 2 GB	Large backups (video, VM images) are lost with no warning; the user discovers it only on recovery.	High
F4	Off-by-one in chunk loop	Cloud copy is truncated and corrupt; a restore returns damaged data.	Critical
F5	Fixed time-out, no resume	Large files on slow networks can never finish and are retried from zero or abandoned.	High
F6	Failure treated as completion	Failed files disappear from the queue; permanent loss with no alert.	Critical
F7	No checksum verification	Corruption (F4) is not detected, so the file is marked Synced.	Critical

4. Equivalence Class Partitioning

4.1 Inferred ECP Used by the Original Developers

The defects show which partitions the developers considered. Their design most likely used coarse partitions of size and type only, with a “normal” network assumed. Each class was tested with one comfortable value (for example 4 KB, 25 MB, 150 MB on a fast link), which is why the suite passed while the real boundaries failed.

Table 6: Inferred original ECP

Factor	Inferred classes	Typical test value	What was not considered
File size	Small (< 100 MB) and Large (≥ 100 MB, up to 2 GB)	4 KB; 25 MB; 150 MB	Zero-byte files, exact chunk multiples, values at or just above 2 GB.
File type	Documents and images; Media and archives; Temporary files (excluded)	.docx; .jpg; .mp4; .tmp	User files that merely resemble temporary names; unsupported or over-long names.
Network speed	Fast and stable only (implicit)	20 to 100 Mbps	Slow, throttled, intermittent and zero-speed conditions.

4.2 Why the Original ECP Failed

- The classes were chosen from the specification’s wording (“small”, “large”) rather than from observed behaviour, so zero, the chunk multiple and the maximum limit were never treated as classes of their own.
- Size, type and speed were partitioned separately. The failures (DR-05, DR-10) occur only when a large size meets a slow network, which a one-factor-at-a-time design cannot reveal.
- No invalid classes were defined for size, so the oversize and corrupt-size behaviours were never exercised.
- Network speed was not partitioned at all, so the time-out logic was never tested.

4.3 Revised Valid and Invalid Equivalence Classes

Table 7: Revised equivalence classes

Class ID	Description	Valid / Invalid	Representative value
S-V1	Empty file	Valid	0 B
S-V2	Small file, 1 B to 1 MB	Valid	4 KB
S-V3	Medium file, above 1 MB and below 100 MB (single upload)	Valid	25 MB
S-V4	Large file, 100 MB to 2 GB, not a multiple of the 10 MB chunk	Valid	105 MB
S-V5	Large file, exact multiple of the chunk size	Valid	100 MB, 200 MB
S-I1	Larger than the 2 GB maximum	Invalid (reject with notice)	2.5 GB
S-I2	File changes or is locked during upload	Invalid (retry, then flag)	Appended while syncing
S-I3	Size cannot be read (corrupt metadata, permission denied)	Invalid (flag)	Unreadable file
T-V1	Documents and text	Valid	.docx, .txt
T-V2	Images	Valid	.jpg, .png
T-V3	Media, archives and disk images	Valid	.mp4, .zip, .iso
T-V4	User file whose name starts with “~” or “.” but is not a real temp file	Valid	~budget.xlsx
T-I1	Genuine temporary or lock file	Invalid for sync (ignore, log)	~$report.docx, .tmp
T-I2	Illegal characters or name longer than 255 characters	Invalid (reject with notice)	256-character name
N-V1	Fast network, 10 Mbps or more	Valid	50 Mbps
N-V2	Moderate network, 1 to below 10 Mbps	Valid	2 Mbps
N-V3	Slow network, 0.25 to below 1 Mbps	Valid	0.5 Mbps
N-I1	Offline or below 0.25 Mbps	Invalid (queue, wait)	0 Mbps
N-I2	Connection drops mid-transfer	Invalid (resume)	Disconnect at 40%
N-I3	High latency or packet loss	Invalid (retry)	500 ms, 10% loss

5. Boundary Value Analysis

Boundary Value Analysis tests the values on and immediately either side of each partition edge, because defects concentrate there. The boundaries below come from the inferred thresholds (100 MB, 10 MB chunk, 2 GB, 300 s, 3 retries) and from the revised classes. Each boundary uses the standard three-point set: just below, on, and just above.

Table 8: Boundary values and expected behaviour

Factor	Boundary	Test values	Expected result
File size	Empty / smallest	0 B, 1 B, 2 B	All synchronized and verified; 0 B is uploaded as an empty object.
File size	Single vs chunked upload	100 MB − 1 B, 100 MB, 100 MB + 1 B	All complete; byte count and checksum match in the cloud.
File size	Chunk multiples	10 MB × n − 1 B, 10 MB × n, 10 MB × n + 1 B (n = 10, 20)	Every chunk including the last is stored; no truncation.
File size	Maximum supported	2 GB − 1 B, 2 GB, 2 GB + 1 B	First two sync; the third is rejected with a visible notice and kept locally.
Network speed	Slow and offline edge	0 Mbps, 0.25 Mbps − ε, 0.25 Mbps, 1 Mbps, 10 Mbps	Offline queues; slow links continue with progress; none cause loss.
Transfer time	Time-out edge	Throttle so that transfer takes 299 s, 300 s, 301 s	Transfer never abandoned; a stall time-out applies only if there is no progress.

Factor	Boundary	Test values	Expected result
Retry count	Retry limit	0, 1, 2, 3 (limit), 4	At the limit the file shows “Failed”, stays in the queue view and is not marked synced.
File name length	Name limit	254, 255, 256 characters	254 and 255 sync; 256 is rejected with a notice.
Concurrent files	Queue capacity	N − 1, N, N + 1 (N = configured queue size)	Overflow items wait; none are dropped.

6. Positive and Negative Test Cases

Table 9: Positive test cases

TC ID	Class / boundary	Input and setup	Expected result
TC-P01	S-V1, FR-03	0 B file on 50 Mbps	Empty object created in the cloud; status Synced.
TC-P02	S-V2, T-V1	4 KB .docx on 50 Mbps	Uploaded in one request; checksum matches.
TC-P03	S-V3, T-V2	25 MB .jpg on 20 Mbps	Uploaded and verified within the target time.
TC-P04	S-V5 boundary	Exactly 100 MB .pdf on 25 Mbps	Ten chunks stored; cloud size is 104,857,600 B.
TC-P05	S-V4, N-V3	105 MB .zip on 0.5 Mbps	Completes in several hours with resume; no abandonment.
TC-P06	T-V4	~budget.xlsx, 12 KB	Treated as a normal user file and synchronized.
TC-P07	Max size	Exactly 2 GB .iso on 100 Mbps	All chunks stored and verified.
TC-P08	N-I2 recovery	Disconnect at 40% of a 150 MB upload, then reconnect	Upload resumes from the last confirmed chunk; no restart.

Table 10: Negative test cases

TC ID	Class / boundary	Input and setup	Expected result
TC-N01	S-I1	2 GB + 1 B file	Rejected with a clear message; local file untouched.
TC-N02	T-I1	~$report.docx and .tmp lock files	Ignored by design; ignore is logged, not shown as an error.
TC-N03	T-I2	File name of 256 characters	Rejected with a notice that explains the limit.
TC-N04	N-I1	Any file with the network off	Queued; status Waiting for network; nothing lost.
TC-N05	Retry limit	Cloud endpoint always returns an error	After 3 retries the file shows Failed and an alert is raised; never marked Synced.
TC-N06	S-I2	File appended while it is uploading	Upload restarts or syncs a new version; no corrupt mix of the two.
TC-N07	S-I3	File with no read permission	Flagged with a permission message; no crash, no deletion.
TC-N08	F4 fault	Upload drops the last chunk deliberately	Checksum mismatch detected; file not marked Synced; retried.
TC-N09	Disk or quota full	Cloud quota exhausted at 99.9%	Clear quota message; remaining queue paused, not discarded.

7. Business Rules and Decision Table

The rules below are reconstructed from the requirements and from the corrected behaviour needed to remove each defect.

Table 11: Reconstructed business rules

Rule	Business rule
BR-01	Only genuine temporary or lock files are excluded; the exclusion is logged.
BR-02	A file of 0 B up to 100 MB − 1 B is uploaded in a single request.
BR-03	A file from 100 MB up to 2 GB is uploaded in 10 MB chunks, including the last chunk.
BR-04	A file larger than 2 GB is rejected and the user is told.
BR-05	A file is Synced only after the cloud size and checksum match the source.
BR-06	A slow or interrupted network never causes abandonment; the transfer waits and resumes.
BR-07	After the retry limit, the file is marked Failed, the user is alerted, and the local copy is retained.
BR-08	A local file is never deleted or overwritten before the cloud copy is verified.

Table 12: Decision table (Y = yes, N = no, – = not applicable, X = action taken)

Conditions / actions	R1	R2	R3	R4	R5	R6	R7
C1 File type is syncable	N	Y	Y	Y	Y	Y	Y
C2 Size is 2 GB or less	–	N	Y	Y	Y	Y	Y
C3 Size is 100 MB or more	–	–	N	Y	N	Y	–
C4 Network is usable	–	–	Y	Y	N	N	–
C5 Retries remain	–	–	–	–	Y	Y	N
A1 Single-request upload			X				
A2 Chunked upload				X			
A3 Verify checksum and mark Synced			X	X			
A4 Queue with back-off					X	X	
A5 Resume from last confirmed chunk						X	
A6 Mark Failed and alert user							X
A7 Notify user of skip or rejection	X	X					X
A8 Retain local copy	X	X	X	X	X	X	X

8. State Transition Model

The lifecycle of each file is modelled with ten states: Idle, Detected, Queued, Uploading, Waiting (for network), Verifying, Synced, Failed, Skipped (rejected with notice) and Conflict. The F6 defect exists because a transition from Failed to Synced was allowed. A correct model forbids it.

Table 13: Valid transitions

From	Event	To	Guard or action
Idle	File created or changed	Detected	Log event.
Detected	Type or size invalid	Skipped	Notify user; retain local copy.
Detected	Type and size valid	Queued	Apply BR-02 or BR-03.
Queued	Network usable	Uploading	Start single or chunked upload.
Queued	Network unusable	Waiting	Back-off timer.
Uploading	Network lost	Waiting	Remember last confirmed chunk.
Waiting	Network restored	Uploading	Resume from last confirmed chunk.
Uploading	All bytes sent	Verifying	Request size and checksum.

From	Event	To	Guard or action
Verifying	Checksum matches	Synced	Update log; now safe to release local lock.
Verifying	Checksum mismatch	Queued	Increment retry count.
Uploading	Server error	Queued	Increment retry count.
Queued	Retry limit reached	Failed	Alert user; keep in queue view.
Failed	User retries or network recovers	Queued	Reset retry count.
Synced	File modified again	Detected	New version cycle.
Uploading	Remote and local both changed	Conflict	Keep both versions.

Table 14: Invalid transitions that must be rejected (negative state tests)

From	Event	Expected system behaviour
Failed	mark_synced without verification	Blocked. The state must stay Failed.
Uploading	Delete local file	Blocked until the state is Synced.
Queued	Checksum verified	Ignored; no upload has happened.
Synced	Chunk acknowledgement received	Ignored and logged as an anomaly.
Skipped	Upload complete	Ignored; a skipped file has no upload.
Waiting	Verification passed	Ignored; nothing was uploaded.
Idle	Remove from queue	Ignored; no queue entry exists.

9. Revised Testing Strategy: BVA Integrated with Stress Testing

The original suite tested each factor in isolation under comfortable conditions. The revised strategy tests the boundaries of each factor and then repeats the boundary values under combined and extreme load. BVA tells us where to test; stress testing tells us what happens when those places are tested under pressure, at volume and for a long time.

9.1 Design Principles

- Every boundary in Table 8 is a mandatory test input, tested below, on and above the edge.
- Size, speed and type are combined, because the failures appear only in combinations (large file with slow link).
- Each test ends with a reconciliation: the number of files, bytes and checksums in the cloud must equal the source.
- Stress is applied at the boundary values, not at typical values, because a boundary under load is the most likely place to lose data.
- A test passes only if no file is lost silently. A visible, explained rejection is acceptable; a silent disappearance is a failure.

9.2 Integrated Boundary Matrix

The matrix combines the size boundaries with the network boundaries. Every cell is a test case and the expected result is shown in the cell.

Table 15: Size boundary × network boundary matrix

File size	0.25 Mbps	1 Mbps	10 Mbps	100 Mbps	Drop mid-transfer
0 B	Synced	Synced	Synced	Synced	Queued then Synced
100 MB − 1 B	Synced (long, resumable)	Synced	Synced	Synced	Resumes, Synced
100 MB	Synced (long, resumable)	Synced	Synced	Synced	Resumes, Synced

File size	0.25 Mbps	1 Mbps	10 Mbps	100 Mbps	Drop mid-transfer
100 MB + 1 B	Synced (long, resumable)	Synced	Synced	Synced	Resumes, Synced
2 GB	Synced (days; resumable)	Synced (long)	Synced	Synced	Resumes, Synced
2 GB + 1 B	Rejected with notice	Rejected with notice	Rejected with notice	Rejected with notice	Rejected with notice

9.3 Stress, Soak and Fault-Injection Scenarios

Table 16: Stress scenarios

ID	Scenario	Load profile	Pass criterion
ST-01	Bulk small files	100,000 files of 0 B to 1 MB, including empty and "~" names	All synced; zero lost; queue drains.
ST-02	Concurrent boundary uploads	200 files at 100 MB − 1 B, 100 MB and 100 MB + 1 B at once	All verified; no truncated copies.
ST-03	Network ramp	100 Mbps down to 0.25 Mbps, then 0 Mbps, then back to 100 Mbps	No abandonment; all resume and finish.
ST-04	Flapping network	Disconnect for 10 s every 30 s for 2 hours	No file lost; retry count behaves as designed.
ST-05	Packet loss and latency	5% to 20% loss, 500 ms latency	Uploads complete; checksums match.
ST-06	72-hour soak	Continuous create, modify, rename and delete workload	No memory growth; zero lost; no stuck queue items.
ST-07	Quota and disk limits	Cloud quota or local disk at 99.9%	Clear error; queue paused, not dropped.
ST-08	Crash recovery	Kill the client mid-chunk, restart, repeat 50 times	Resumes with no duplicate or missing chunk.
ST-09	Hot files	Files modified during upload, 1,000 times	Final cloud version equals final local version.
ST-10	Maximum-size parallel load	10 clients each uploading a 2 GB file	All verified; server rate limits handled with back-off.

9.4 Test Execution Plan

Table 17: Execution phases

Phase	Activity	Technique / evidence	Acceptance criterion
1. Log analysis	Mine real logs and defect reports for size, type and speed patterns.	Pattern analysis (Table 4)	Every lost file is explained by a hypothesis.
2. Design	Build revised ECP and BVA sets and the decision table.	ECP, BVA, decision table	All classes and boundaries have test cases.
3. Functional boundary run	Execute Tables 8, 9, 10 and 15 on a controlled network.	Black-box testing	All boundary tests pass.
4. Fault injection	Drop the last chunk, force time-outs and server errors.	Negative testing	Faults are detected and never marked Synced.
5. Stress and soak	Run ST-01 to ST-10.	Load generator and network shaping	Zero lost files; no resource exhaustion.
6. Reconciliation audit	Compare source and cloud manifests (count, bytes, checksum).	Automated audit script	100% match.
7. Regression in CI	Run boundary and smoke stress tests on every build and release.	Automated pipeline	No fixed defect reappears.

9.5 Instrumentation and Safeguards

- Log every state change with file ID, size, chunk index, retry count and network speed measured.
- Emit metrics for queue depth, number of files per state and time spent in Waiting or Failed.
- Run a periodic reconciliation job that compares local and cloud manifests and raises an alert on any difference.
- Replace the fixed 300 s total time-out with a progress-based stall time-out and chunk-level resume.
- Mask file names or content in logs where privacy is required.

10. Justification: How the Reconstructed Approach Prevents Data Loss

The reconstructed approach prevents data loss because each cause of loss identified in Section 3 is matched by a test that can detect it, a rule that forbids it, and a runtime control that stops it from going unnoticed.

Table 18: Defect-to-prevention traceability

Fault	Revised test that exposes it	Rule or safeguard that prevents it
F1 Wide temp filter	TC-P06, TC-N02 (T-V4 vs T-I1)	BR-01: exclude only genuine temp patterns; log every exclusion.
F2 Empty file skipped	TC-P01, BVA 0 B, 1 B	BR-02: 0 B is a valid size and is uploaded.
F3 Silent size skip	TC-N01, BVA 2 GB ± 1 B	BR-04 and A7: rejection must notify the user.
F4 Off-by-one chunk	TC-P04, chunk-multiple BVA, ST-02	BR-03: last chunk is always sent; test covers 10 MB × n.
F5 Fixed time-out	TC-P05, ST-03, ST-04	BR-06: progress-based stall time-out and resume.
F6 Failure marked done	TC-N05, invalid transitions in Table 14	BR-07: Failed state with alert; Failed to Synced is blocked.
F7 No verification	TC-N08, reconciliation audit	BR-05: size and checksum must match before Synced.

Four properties make the approach reliable. First, boundaries are tested explicitly, so the edge cases that failed in production are now part of the permanent suite. Second, combinations are tested, so a large file on a slow network is no longer an untested situation. Third, stress and soak runs reveal problems that appear only with volume, concurrency, long duration or crashes. Fourth, BR-05 and BR-08 make loss structurally unlikely because a local file is never released until the cloud copy is verified, and the reconciliation audit would reveal any remaining discrepancy.

Table 19: Recommended coverage targets

Metric	Target	Reason
Equivalence class coverage	100% of valid and invalid classes	Every partition in Table 7 is exercised.
Boundary value coverage	100% of boundaries (below, on, above)	Edge defects are the main cause of loss.
Decision-table rule coverage	100% of rules R1 to R7	Every business rule outcome is checked.
State transition coverage	100% valid, 100% invalid transitions	Prevents illegal moves such as Failed to Synced.
Reconciliation match rate	100% of files, bytes and checksums	Direct measure of zero data loss.
Silent-loss defects	Zero open before release	Silent loss is a release-blocking failure.
Regression reproducibility	Every field defect has an automated test	Prevents recurrence.

11. Challenges and Limitations

- Assumed evidence: the defect reports and thresholds are assumptions. The real values must be taken from actual logs and the product specification, and the ECP and BVA tables updated accordingly.
- Black-box inference: the pseudocode is the simplest explanation that fits the observations. Several different implementations could produce the same symptoms, so code review should confirm it.

- Test environment cost: 2 GB files on a 0.25 Mbps link take days. Network emulation, time acceleration and sampling are needed to keep the schedule realistic.
- Cloud provider limits: rate limiting, quotas and service outages can look like client defects and must be separated during analysis.
- Platform variation: file systems differ in name rules, case sensitivity, locking and symbolic links, so tests must repeat on each supported operating system.
- Concurrency and clocks: multi-device editing and clock differences create conflict cases that need dedicated testing.
- Flaky results: network tests can fail for external reasons, so every failure must be reproduced and classified before it is recorded as a defect.

12. Conclusion and Future Scope

The lost-file reports are not random. They follow the boundaries of file size, the limits of the time-out on slow networks, the over-broad type filter and the treatment of failures as completion. Reconstructing the likely algorithm shows that the original ECP was coarse, considered only typical values, and ignored network speed, so it could not reveal these faults.

The revised approach replaces it with a richer set of valid and invalid equivalence classes, explicit boundary values, a decision table for the business rules, a state model that forbids unsafe transitions, and a stress programme run at the boundary values. Together with verification before completion, visible failure reporting and a reconciliation audit, it targets each identified cause of loss and provides measurable evidence that no file is lost silently.

In future, the tool can adopt property-based testing that generates random sizes and network profiles, chaos testing in production-like environments, continuous reconciliation dashboards, and machine-learning anomaly detection on activity logs to catch unusual loss patterns early.

12.1 References

[1] Glenford J. Myers, Corey Sandler and Tom Badgett, The Art of Software Testing, 3rd ed., John Wiley & Sons, 2011.

[2] Boris Beizer, Software Testing Techniques, 2nd ed., Van Nostrand Reinhold, 1990.

[3] Roger S. Pressman and Bruce R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed., McGraw-Hill, 2020.

[4] Paul C. Jorgensen, Software Testing: A Craftsman's Approach, 4th ed., CRC Press, 2014.

[5] IEEE, IEEE Standard for Software and System Test Documentation, IEEE Std 829-2008, 2008.

[6] ISO/IEC/IEEE, Systems and Software Engineering – Software Testing, ISO/IEC/IEEE 29119 series.

[7] ISTQB, Certified Tester Foundation Level Syllabus, International Software Testing Qualifications Board.

[8] L. J. White and E. I. Cohen, "A Domain Strategy for Computer Program Testing," IEEE Transactions on Software Engineering, vol. SE-6, no. 3, pp. 247–257, 1980.

[9] T. J. Ostrand and M. J. Balcer, "The Category-Partition Method for Specifying and Generating Functional Tests," Communications of the ACM, vol. 31, no. 6, pp. 676–686, 1988.

Code of Conduct:

I, M Sandhya(192312421) , certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Sandhya**

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements.	Clearly understands the scenario with minor gaps.	Basic understanding; some aspects of the scenario are unclear.	Poor understanding or misinterpretation of the scenario.	
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems.	Applies relevant concepts correctly with minor errors.	Applies basic concepts but with limited accuracy.	Fails to apply appropriate concepts.	
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification.	Good analysis with reasonable solutions.	Limited analysis; solutions lack justification.	Poor or no analysis; incorrect decisions.	
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach.	Logical and structured solution with minor gaps.	Basic solution with limited structure or feasibility.	Unclear or incorrect solution approach.	
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning.	Clear explanation with some justification.	Limited clarity and weak justification.	Poorly explained with no justification.	